

P0986R0

Comparison of Modules Proposals

Published Proposal, 9 March 2018

This version:

<http://wg21.link/p0986r0>

Authors:

[Richard Smith](#) (Google)

[David Jones](#) (Google)

Audience:

EWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Abstract

This paper provides a comparison of the modules design in P0947R0 to the Modules TS.

Table of Contents

1	Overview
1.1	The textual inclusion problem
2	Source file layout
3	Semantic export
3.1	Reachability of exports
3.2	Exporting just a declaration
3.3	Avoiding writing unnecessary data
4	Name lookup
4.1	Two-phase lookup and ADL

5	Module partitions
6	Of Modules and Macros
6.1	Proposal updates
6.2	Modularity
6.3	Representing modules
6.4	Exporting macros
7	Splitting the Atom

References

Informative References

§ 1. Overview

This companion paper to [\[P0947R0\]](#) ("**Another take on modules**", hereafter called the Atom proposal) provides a comparison of the design in that paper to the design of the Modules TS ([\[N4719\]](#)), focusing on examples and concerns that have been raised with the Modules TS and showing how the design in [\[P0947R0\]](#) might address them.

§ 1.1. The textual inclusion problem

Most of the problems we would like to see solved by modules are fundamentally caused by textual inclusion: lack of interface isolation, interface fragility, slow builds, ODR violations, and so on, are unintended consequences of the fact that interfaces are made visible by textually including the contents of a header file.

However, we would be remiss to ignore the likelihood that legacy headers will continue to be common for several versions of the C++ standard after a modules system is adopted -- at least a decade. In some cases, we expect they will continue to exist for the foreseeable future of the C++ language. A modules system -- at least, one that seeks to fully solve the problems resulting from textual inclusion -- must provide some way to use the interfaces provided by legacy headers, without reprising the unintended consequences of textual inclusion.

The Modules TS goes a long way towards solving the textual inclusion problem. However, because the Modules TS's only solution for use of legacy headers is textual inclusion, it does not completely solve the problem. The Atom proposal extends modular semantics to legacy headers, without exposing modular code to the unintended consequences of textual inclusion. This is achieved by extracting the

interface provided by the header and making it available for semantic import, allowing new code to have a complete solution to the textual inclusion problem.

EXAMPLE 1

Modules TS

```
// textual header, expanded
// by preprocessor
#include "legacy-header.h"
module Foo;
```

P0947R0

```
module Foo;
// semantic import of interface provided
// by the textual header, as if by
// 1) #including it in a separate context
// 2) extracting the interface
// 3) importing that interface
import "legacy-header.h";
```

The use of textual inclusion for legacy headers in the Modules TS results in a number of issues. For instance:

- Compile time problems are not fully solved. For as long as legacy headers remain common, we will continue to repeatedly parse and compile the same interfaces over and over again within a build. And worse, in highly parallel builds, we now need to wait for N levels of module interface parsing before we can start building leaf code; if each such level is doing a comparable amount of textual inclusion to today's compilations, our highly-parallel build is now N times slower. Previous attempts to address this cost, such as Precompiled Headers (PCH), still have drawbacks that would not be addressed by modules.
- Macro and name leakage problems remain. Without the rules and semantics required for modular interfaces, names (including macro names) would continue to leak between legacy headers, even within new, modular code. As a result, there would still be a risk of ODR violations in modular code, due to interfaces textually included from legacy headers. Additionally, the design of the Modules TS forces all legacy header inclusions to be before the module declaration, so that even the *module-declaration* and *import-declarations* risk being modified by macros from legacy header. Correctly determining the dependencies of a source file, and even the module name, requires a full preprocessing step. The Atom proposal implicitly forms a modular interface

boundary around the header itself, so that the import of the legacy header has a semantic effect, rather than a textual one, on the new, modular code, and isolates the *module-declaration* and *import-declarations* from macros in legacy headers.

- Faithfully exporting the complete contents of a module interface unit that textually includes legacy headers could still become prohibitively expensive for large volumes of transitive textual inclusion. An implementation must therefore be selective in which portions of a legacy header it writes to its Binary Module Interface (BMI) file -- blindly emitting all declarations would result in BMIs containing highly-redundant duplicate information from legacy header files. This circles back to the leakage problem of textual inclusion, and forces compromises to be made regarding visibility of names and semantic effects.

The Atom proposal avoids these problems by excising textual inclusion of legacy headers from modular code, instead providing synthesized modules for textual headers, which are directly usable by its module system. Under the Atom proposal, legacy (but nonetheless modular) headers need be parsed only once in a complete compilation, and their modular representation can be shared by all compilations making use of the legacy header's interfaces. The interface provided by the legacy header is completely isolated from undesirable macro and name leakage problems.

EXAMPLE 2

To see some of the differences in how the Modules TS and the Atom proposal handle code in textual headers, consider a program with three translation units -- two modules and one main file -- expressed in a form compatible with the Modules TS, and a form compatible with the Atom proposal:

Modules TS

```
#include "logging.h"
export module Foo;
export void f() {
    LOG(INFO) << "...";
}
```

```
#include "other.h"
#include "logging.h"
export module Bar;
import Foo;
export void g() {
    LOG(INFO) << "...";
    f();
}
```

```
#include "logging.h"
import Bar;
int main() {
    LOG(INFO) << "...";
    g();
}
```

P0947R0

```
export module Foo;
import "logging.h";
export void f() {
    LOG(INFO) << "...";
}
```

```
export module Bar;
import "other.h";
import "logging.h";
import Foo;
export void g() {
    LOG(INFO) << "...";
    f();
}
```

```
import Bar;
import "logging.h";
int main() {
    LOG(INFO) << "...";
    g();
}
```

In the Modules TS version of this example, some problems are evident:

- In order to compile the main translation unit, we must wait for `logging.h` to be textually included and parsed three times, sequentially, potentially increasing the time between a developer initiating a build and getting back a binary.
- The result of including, preprocessing, and semantically analyzing the `logging.h` header in module `Bar` may be different than in module `Foo`, because names and macros from `other.h` are visible when `logging.h` is parsed in `Bar`.

Under the Atom proposal, these problems are avoided. The header `logging.h` is notionally only `#included` once, into a separate translation unit, from which its interface is extracted and then made available to `Foo`, `Bar`, and the main source file.

§ 2. Source file layout

Under the Modules TS, the *module-declaration* specifying that source file is in fact part of a module may appear arbitrarily far through the file. This makes it hard, for example, for a human reader or for a tool (such as a build system) to determine essential facts about a source file, such as whether it is part of a module and what other modules it depends on (see [\[P0273R0\]](#), [\[P0629R0\]](#), [\[P0713R1\]](#) for prior discussion). Under the Atom proposal, the *module-declaration* and all *import-declarations* appear in a *preamble*, at the beginning of the implementation file, prior to any other top-level declarations.

EXAMPLE 3

Modules TS

```
#include "foo.h"
#include "bar.h"
/* global module declarations */
export module M;
/* implementation */
import frog;
import badger;
/* more implementation */
```

P0947R0

```
export module M;
import "foo.h";
import "bar.h";
import frog;
import badger;
/* implementation */
```

The ability to delay the *module-declaration* is necessary only to support textual inclusion of legacy headers and the injection of "prefix headers" -- such constructs are written prior to the *module-declaration*, outside the "purview" of the module. Under the Atom proposal, all interface importation, whether from modularized code or from legacy headers, is expected to be performed by semantic

import, so the *module-declaration* can simply be required to appear first; by excising textual inclusion and preprocessing from new modular code, there is no need for a notion of "purview."

The Atom proposal's *preamble* is not only useful for human readers to determine the imports of a file, but also for tools to automatically insert and manage imports, and for build systems to infer module dependencies and form a correct dependency graph and build module interfaces in the correct order. Under the Modules TS, the entirety of a source file must be examined to correctly handle such cases.

This source file layout choice allows us to avoid a problem that the Modules TS has run into: the introduction of the `module` and `import` keyword is a major breaking change for several codebases that use these keywords as identifiers. For example:

- In Google's codebase, there are around 10 uses of `import` and 30 uses of `module` per million lines of code.
- In the LLVM codebase, there are uses of the identifier `module` in interfaces with hard API stability guarantees.
- [\[P0795R0\]](#) identifies the public API of the Vulkan library and many others as being broken by the `module` keyword.

As the `module` and `import` keywords are only used in the preamble portion of a source file under the Atom proposal, they can be (and are) made context sensitive keywords that are only recognized within the preamble. It is thought that it might be possible to make at least the `module` keyword be context-sensitive in the Modules TS ([\[P0924R0\]](#)), but doing so introduces a need for disambiguation rules and attendant language complexity.

§ 3. Semantic export

The problems associated with textual inclusion can be viewed in terms of how much is exported by an interface -- or, more saliently, how much extra is exported that is only ancillary to the interface itself. Under textual inclusion, this volume is often simply too much. A workable modules system should be precise in what it exports: we want to avoid pollution of names, so the module should limit the names it exposes; however, interpreting these names generally requires additional semantics.

§ 3.1. Reachability of exports

When a name is exported, we need to know which of its semantic properties (and those of any other names reachable through it) are also exported. The set of semantic properties considered here is broad, and includes, for example:

- whether a class is defined,
- whether a function parameter has a default argument
- whether an array variable has a declared bound,

and so on. The rules in the Modules TS for specifying which semantic properties are exported by an *export-declaration* will be briefly summarized below. (Note that this summary is not complete, by any means; the actual wording for these rules is more complex, comprising around 20% of the full text of the TS document by page count.)

The central notion in use by the Modules TS is that of reachability at the point of export: when a declaration is exported, all semantic properties that can be found by starting at that declaration and walking through the entities referenced by it are also exported (regardless of whether those referenced entities are themselves explicitly exported). There is also a consistency rule: if two exports result in two different sets of semantic properties being considered reachable for the same entity, the program is ill-formed.

EXAMPLE 4

Modules TS

```
export module M;
struct A;
struct B { A *p; int n; }; // definition implicitly exported by export
export struct A { B *p; };
```

Here, the export of `struct A` results in the definition of `struct B` being "reachable" from importers of module `M`:

```
import M;
int f(A *a) { return a->p->n; } // OK, definition of B reachable
```

However, the name `B` is not visible to importers of module `M`, because it is not exported.

This rule creates a readability and maintenance problem: the semantics of the program depend deeply and subtly on the source file order, and an innocent-looking source reordering can have unexpected effects on the interface exported by a module, by increasing or reducing the set of exported semantic properties or by rendering the program ill-formed.

EXAMPLE 5

Modules TS

A maintenance programmer may decide to rearrange the contents of the module interface unit from the previous example as follows:

```
export module M;
struct A;
struct B; // state of B at this point exported by export of A
export struct A { B *p; };
struct B { A *p; int n; }; // definition no longer implicitly exported
```

This appears to be an innocent change. The module interface unit still compiles. But the client code is now broken, because the definition of `struct B` is no longer reachable at the point of the `export`.

And seemingly-irrelevant changes can have surprising effects on the program. Notably, the rule for consistency of exports may render a program ill-formed simply due to the order in which declarations appear in the modular file.

EXAMPLE 6

Modules TS

Suppose after further development work, the module interface looks like this:

```
export module M;
struct A;
struct B; // state of B at this point exported by export of A

export struct A { B *p; };
struct B { A *p; int n; }; // state of B at this point exported by X::make
export class X {
    X(...);
private:
    B *make() { ... }
};
```

Because struct `B` had no definition when referenced from the definition of exported class `A`, the later implicit export of the definition of struct `B` by the private member function `X::make` results in `M` being ill-formed.

Presuming that the intent is to not export the definition of `B`, `M` would need to be carefully restructured; for example, like this:

```
export module M;
struct A;
struct B; // state of B at this point exported by A and X::make

export struct A { B *p; };
export class X {
    X(...);
private:
    B *make();
};
struct B { A *p; int n; };
inline B *X::make() { ... }
```

Note that this adds another source-layout constraint: now the `X::make` function must be moved out of line.

The Atom proposal uses a different rule for visibility that does not rely on source ordering:

- name visibility is restricted to those names exported by the module
- all semantic effects within the module interface unit are visible to a direct importer of that translation unit

This tradeoff affords module authors precise control over which parts of the module interface can be accessed by an importer -- a notion of narrowed reachability -- while also ensuring that uses of the exported interface are uniform and unambiguous.

EXAMPLE 7

P0947R0

In the Atom proposal, the module interface from the prior example can be represented as follows:

```
export module M:BImpl; // partition BImpl of module M
struct A;
struct B { A *p; int n; };

export module M;
import :BImpl; // import of partition BImpl, without re-export
export struct A { B *p; };
export class X {
    X(...);
private:
    B *make() { ... }
};
```

Note that the relative order of declarations has no effect on which names and semantic effects are exported. The definition of `B` is not exported from `M` because `M` does not export the partition `M:BImpl` in which it is defined.

§ 3.2. Exporting just a declaration

One of the goals of the Modules TS' reachability model is to permit control over whether a definition or merely a declaration of an entity is exported, but as shown, the technique used is fragile in the presence of source reordering. The module partitions feature in the Atom proposal provides a mechanism to control which semantic effects are exported in a clear way that is robust against source reordering.

EXAMPLE 8

Suppose we wish module `M` to export an opaque type `Handle`, which is a pointer to some implementation type `Impl`, along with some inline functions that access the `Impl` object.

Modules TS

```
export module M;
// Export Handle as a typedef
// for a pointer to incomplete
// type Impl
struct Impl;
export using Handle = Impl*;

// The definition of Impl must
// appear after the export of
// Handle or the definition will
// also be exported
struct Impl { int n; };
export inline int get(Handle h) {
    return h->n;
}
// ...
```

P0947R0

```
// Definition of struct Impl is
// owned by the Impl partition
// of module M
export module M:Impl;
struct Impl { int n; };
```

```

export module M;
// Semantic effect of defining
// struct Impl is not re-exported
import :Impl;

// Export Handle as a typedef
// for a pointer to type Impl,
// which is incomplete outside
// module M
export using Handle = Impl*;
export inline int get(Handle h) {
    return h->n;
}
// ...

```

In the Atom approach, the non-exported semantic properties are in a distinct source file. (The downside is that this level of control requires at least one additional source file to exist, but the presence of module partitions is, by design, not visible outside the module.)

Note that the name of struct `Impl` is visible within `M` despite not being exported from `M:Impl`. An import of a translation unit in the same module makes available all names from that translation unit, whether exported or not. The definition of `struct Impl` is visible within `M` because the translation unit `M:Impl` containing its definition is imported. However, it is not visible to translation units outside module `M` that import `M`, because `M` does not re-export `M:Impl`. Further, `M:Impl` cannot be directly imported into translation units outside module `M`.

§ 3.3. Avoiding writing unnecessary data

A desirable property present in the Modules TS rule is that it allows an implementation to be more frugal with the set of data it exports to the BMI: it can start at the set of exported declarations and recurse through its internal representation, writing out only those things that are reachable, and not need to emit any other data. This is still possible under the Atom proposal, but it is now a quality-of-implementation optimization, rather than forming part of the language semantics and being included in the user model. An implementation wishing to perform this optimization would need wait until the end of the translation unit to perform the traversal of its internal representation, to ensure it does not miss semantic properties added after the point of export.

§ 4. Name lookup

§ 4.1. Two-phase lookup and ADL

Name visibility is complicated by deferred forms of name lookup. There are two widely-used techniques: two-phase lookup of class (and enumeration) member names, and argument-dependent name lookup (ADL). In both cases, the binding of a name can be deferred from the point of template definition until to the point of instantiation, and the programmer may intend for names from either context (or an intervening context) to be visible.

In the Modules TS, names from the following sources are considered when template instantiation requires name lookup:

- names from the (outermost) point of instantiation and from the (innermost) point of definition (names from translation units containing intervening instantiations are ignored)
- for ADL involving an associated type owned by a named module, names from the module interface unit owning the type

The Atom proposal considers these sources:

- names from the (outermost) point of instantiation, names from the (innermost) point of definition, and names from every intervening module interface along the path of instantiation

ADL involving an associated type owned by a named module is not treated as a special case. We are open to revisiting this, but the motivating examples for the rule in the Modules TS involve a type that is visible along the path of instantiation, so it is unclear that motivation for a special case rule remains when using the lookup rule from the Atom proposal.

The following examples, taken from **[temp.dep.res]** in the Modules TS, illustrate the difference:

EXAMPLE 9

Modules TS

```
// Header file X.h
namespace Q {
    struct X { };
}
```

```
// Interface unit of M1
#include "X.h" // global module
namespace Q {
    void g_impl(X, X);
}
export module M1;
export template<typename T>
void g(T t) {
    g_impl(t, Q::X{ }); // #1
}
```

```
// Interface unit of M2
#include "X.h"
import M1;
export module M2;
void h(Q::X x) {
    g(x); // OK
}
```

P0947R0

```
// Header file X.h
namespace Q {
    struct X { };
}
```

```

export module M1;
import "X.h";
namespace Q {
    extern "C++" void g_impl(X, X);
}
export template<typename T>
void g(T t) {
    g_impl(t, Q::X{ }); // #1
}

```

```

export module M2;
import "X.h";
import M1;
void h(Q::X x) {
    g(x); // OK
}

```

In this example, the Modules TS and the Atom proposal give the same outcome for different reasons. In the Modules TS, we rely on lookup at #1 in the template definition context to find `Q::g_impl` despite it not being visible to normal lookup. In the Atom proposal, `Q::g_impl` is also visible to lookup in the instantiation context, because `M1` is in the path of instantiation.

The Modules TS creates an asymmetry between types owned by modules, and those from legacy headers. In some cases, this results in a failure to find names that were intended to be found. Fixing this is problematic in the Modules TS: because legacy headers are textually included at every point of use (see [§1.1 The textual inclusion problem](#)), it would be prohibitively expensive to retain duplicate copies of all declarations from all legacy headers in case they are reached by a template instantiation.

Because the Atom proposal performs separate modular compilation even for legacy header files, sufficient information can be cheaply retained to permit a consistent semantic model even when mixing legacy code with modular code. The same rules apply to the availability of semantic properties, such as type completeness, which the Modules TS does not appear to address.

EXAMPLE 10

Modules TS

```
// Module interface unit of A
export module A;
export template<typename T>
void f(T t) {
    t + t; // #1
}
```

```
// Module interface unit of B
export module B;
import A;
export template<typename T, typename U>
void g(T t, U u) {
    f(t);
}
```

```
// Module interface unit of C
#include <string>
import B;
export module C;
export template<typename T>
void h(T t) {
    g(std::string{ }, t);
}
```

```
// Translation unit of main()
import C;
void i() {
    // ill-formed: '+' not found at #1
    // nor at point of instantiation of
    // f<std::string> (just before 'i()')
    h(0);
}
```

```

export module A;
export template<typename T>
void f(T t) {
    t + t; // #1
}

```

```

export module B;
import A;
export template<typename T, typename U>
void g(T t, U u) {
    f(t);
}

```

```

export module C;
import <string>;
import B;
export template<typename T>
void h(T t) {
    g(std::string{ }, t);
}

```

```

// Translation unit of main()
import C;
void i() {
    // OK: path of instantiation includes
    // module interface units of A, B, C,
    // and point of instantiation of
    // h<int> (just before 'i()');
    // operator+(string, string) visible
    // in C
    h(0);
}

```

The Atom proposal accepts this example that the Modules TS rejects. Under the Modules TS, it would be prohibitively expensive to retain a full copy of `<string>` as part of the BMI for module `C`, and there is no hint in the definition of `C` that `string`'s `operator+` will be needed. As such, it is not available, although no translation unit in this example did anything unreasonable.

Under the Atom proposal, an external modular representation of `<string>` is available, independent of module `C`, and the compiler can reference that when performing the instantiation of `f<string>` with `C` in the path of instantiation.

ADL idioms, such as `using std::min; min(...)`, would be affected when the set of scopes considered for lookup is changed. Consider this example:

EXAMPLE 11

Suppose module **M1** exports a template function. Further, suppose module **M2** imports **M1**, and exports a different template function which calls **M1**'s exported function:

Modules TS

```
// Module interface unit of M1
#include <algorithm>
export module M1;
export template<typename T, typename U>
void f(T& t, U& u) {
    min(t, u); // #1
}
```

```
// Module interface unit of M2
#include <locale>
struct Aux : std::ctype_base {
    operator int() const;
};
void min(Aux&, Aux&); // #2
export module M2;
import M1;
export template<typename T>
void g(T t) {
    Aux aux;
    f(aux, aux);
}
```

```
// Elsewhere, translation unit
// of global module
import M2;
void h() {
    g(0);
}
```

P0947R0

```

export module M1;
import <algorithm>;
export template<typename T, typename U>
void f(T& t, U& u) {
    min(t, u); // #1
}

```

```

export module M2;
import M1;
import <locale>;
// Note: extern "C++" denotes
// non-modular ownership semantics
extern "C++" {
    struct Aux : std::ctype_base {
        operator int() const;
    };
    void min(Aux&, Aux&); // #2
}
export template<typename T>
void g(T t) {
    Aux aux;
    f(aux, aux);
}

```

```

// Elsewhere, not in
// a module
import M2;
void h() {
    g(0);
}

```

As described in [\[temp.dep.res\]p5](#) of the Modules TS, the Modules TS version of this program has undefined behavior. The behavior in the Atom proposal is defined: when the call to `min` within `f<Aux, Aux>` at #1 is instantiated, the path of instantiation starts in `h`, includes the module interface unit of `M2`, and ends in `f` (which considers the module interface unit of `M1`). For this path of instantiation, the `min` declared at #2 is a candidate and is selected.

§ 5. Module partitions

The desire for a module partition system has been explored in [\[P0273R0\]](#), [\[P0584R0\]](#), and [\[P0775R0\]](#). The module partition system in the Atom proposal is derived from the system proposed in [\[P0775R0\]](#), which saw strong support at Albuquerque. We believe this partition system would be equally effective as an addition to the Modules TS.

§ 6. Of Modules and Macros

The paper [\[P0955R0\]](#) provides valuable insights into the notion of modularity and what a modules system for C++ should provide, and we would like to analyze its arguments and compare how they apply to the Modules TS and to the Atom proposal.

§ 6.1. Proposal updates

In response to the commentary in [\[P0955R0\]](#), as well as private feedback, we have made two changes to the direction in [\[P0947R0\]](#):

- When a legacy module unit is re-exported by a module interface unit, only its names and semantic effects are re-exported; its macros are not. This makes it trivial to form a "macro-free" modular version of a legacy header:

```
export module my.module;  
export import "my/header.h";
```

- The `#export` syntax is explicitly noted as being a separable part of the proposal, on which no other part depends. We still believe this functionality is important for the medium-term future of C++ (until the use cases requiring macros as part of interfaces are addressed in other ways), but its acceptance or otherwise need not be coupled to the rest of the proposal.

The consequence of these two changes is that, in the case where `#export` is not adopted, only legacy modules can contain macros and only legacy module imports can import macros, allowing a user to visually distinguish between an import-with-macros and an import-without-macros. Compared to the Modules TS, the only places where an `import` provides macros would be the same places where the Modules TS would require a `#include`.

§ 6.2. Modularity

As [P0955R0] explains, code hygiene requires that we avoid textual inclusion for interface import, specifically because earlier names and macros "leak" into later textually-included interfaces. Semantic import, such as that offered by the `import` facilities of the Atom proposal or the Modules TS, avoids these problems by defining the interface of the imported component independent of the accumulated state at the point of import.

Because the Modules TS relies on textual inclusion for legacy headers, the non-modularity problems inherent to textual inclusion will still occur for as long as legacy headers remain.

EXAMPLE 12

Modules TS

```
#include "legacy1.h"
#include "legacy2.h"
module M;
import canines.sam;
import lagomorphs.max;
```

P0947R0

```
module M;
import "legacy1.h";
import "legacy2.h";
import canines.sam;
import lagomorphs.max;
```

In the Modules TS, names and macros from `"legacy1.h"` still leak into `"legacy2.h"`. In the Atom proposal, because each legacy header is (notionally) compiled in isolation, names and macros from the importing translation unit cannot leak from one legacy header to another.

In the Modules TS, macros from `"legacy1.h"` and `"legacy2.h"` can even affect the interpretation of later `import`s: if `"legacy1.h"` exports an object-like macro named `max`, this macro will intrude upon the `import lagomorphs.max;` line later and alter the imported module name. This means that a build system needs to perform a full preprocessing step (including textual inclusion of legacy headers) to determine the build dependencies for `M`. Under the Atom proposal, this cannot happen: expansion of macros from legacy headers is disallowed within the preamble of the source file. In order to make this possible, imported macros are distinguished from those on the command line, and a natural source ordering of imports is enforced. (Under the current proposal, such expansion is ill-formed, as we find that to be the more obvious behavior, but we could equally specify that such macro expansion simply does not occur.)

§ 6.3. Representing modules

The description in [\[P0955R0\]](#) of macros describes a concern with macros whose definitions differ between translation units:

file 1:

```
#define Foo 1
// define module M
```

file 2:

```
#define Foo 2
import M;
// use M and Foo
```

[\[P0955R0\]](#) observes that the code in file 1 and file 2 will see a different value of `Foo` (perhaps `Foo` is `NDEBUG`), and suggests that this will lead to disaster if, for example, a `struct` used by both file 1 and file 2 (say, `std::vector<T>`) depends on the value of `Foo`.

However, this argument has an implicit, intervening step: in order for file 1 and file 2 to see different versions of the `struct`, the `struct` definition must have been compiled twice, (presumably) by textual inclusion. That is: the problem highlighted here is a consequence of the use of textual inclusion to provide a definition of the `struct`. Under the Atom proposal, the disaster scenario outlined by [\[P0955R0\]](#) does not occur:

EXAMPLE 13

For the purpose of this example, let us assume that the standard library has not yet been modularized, and is being treated as a legacy header. Obviously, even once the standard library is provided as a set of modules, the same situation will continue to arise with other legacy headers.

Modules TS

```
#undef NDEBUG
#include <vector>
export module M;
export std::vector<int> f() {
    return {1, 2, 3};
}
```

```
#define NDEBUG 1
#include <vector>
#include <iostream>
import M;
int main() {
    for (int n : f())
        std::cout << n;
}
```

P0947R0

```
#undef NDEBUG
export module M;
public import <vector>;
export std::vector<int> f() {
    return {1, 2, 3};
}
```

```
#define NDEBUG 1
import <vector>;
import <iostream>;
import M;
int main() {
    for (int n : f())
        std::cout << n;
}
```

Let us suppose that `std::vector` uses a different representation when `NDEBUG` is defined. Under the Modules TS, the program will likely crash, due to differing definitions of the `vector` type. Under the Atom proposal, the program is valid -- the legacy header `<vector>` is processed in an environment that is independent from its point of import, and so both translation units share the same semantic model for `<vector>` and thus the same representation for `std::vector<int>`.

This problem can be solved without making the sacrifices that [\[P0955R0\]](#) warns us about (switching between different module interfaces based on macros defined prior to the import, or bifurcating the BMI for a module at points affected by macros), and indeed those approaches do not solve the problem in the above example. The problem is caused by repeated textual inclusion of the same legacy header into environments where it is interpreted in different ways, and is solved by avoiding that textual inclusion.

§ 6.4. Exporting macros

[\[P0955R0\]](#) presents a question. Given:

```
import M;  
// ...
```

If `M` can export macros, then how are those macros to be discovered when later code mentions them? How would the compiler know to look into `M`? (Would this mean that `import` is no longer "free"?)

It should be noted that an analogous question applies for non-macro names exported for modules. How is a compiler to know whether *any* identifier it sees after a module import should resolve to a name from within that imported module? The answer is the same in both cases: when a compiler sees an identifier, it must check (ideally with some caching) whether that identifier is declared in an imported module. This requirement is not unique to macro export.

As noted in [\[P0955R0\]](#), macro export and import requires preprocessor changes. However, much like the question of lookup, the question of import and export is analogous to imports and exports of modular declarations.

Because modules are `import`ed, and not `#included`, it follows directly from the phases of translation that recursive preprocessing is not performed at import time -- the `import` declaration is not a preprocessor directive. Instead, because the Atom proposal limits `import`s to the preamble, the preprocessor can simply load the macro definitions exported by each of the imported modules, and there can be no interaction between the imports. The preprocessor can later emit any exported macros at the same time as the Binary Module Interface. (Note that macro definitions do not necessarily need to be stored in the BMI, and could equally be stored as a separate file alongside the BMI.)

Also of note: as described above, the order-dependence problem mentioned in [\[P0955R0\]](#) is a problem under the Modules TS when using its support for legacy headers. It is not a problem under the Atom proposal, as we can detect and diagnose uses of imported macros within *import-declarations*.

§ 7. Splitting the Atom

The Atom proposal provides a coherent modules design that is some distance from the Modules TS along various axes. It is natural to ask whether we can incrementally add some of these features to the Modules TS, or whether adopting them would reasonably need to be a "step change".

There are various dependencies between these changes; some of these are:

- The Atom proposal's name visibility rule depends on module partitions (in order to provide a mechanism for controlling which semantic effects are imported), and also -- to some extent -- on legacy header units (in order to allow that control to extend to declarations from legacy headers)

- Making `import` and `module` context-sensitive depends, at least partially, on the required source file order change.
- The required source file order change depends on the ability to acquire the interface of legacy headers after the *module-declaration*; in the Atom proposal, this is made possible by legacy header import.
- The ability to correctly build legacy header units from existing code depends on the ability of other legacy header units to export macros.

However, several parts of this proposal are separable:

- Support for module partitions does not depend on any other part of this proposal, and could be directly adopted by the Modules TS. This has already been proposed separately ([P0775R0]).
- The ability for translation units (other than legacy header units) to export macros is a separable feature. Macro export for legacy header units is necessary in order to retain functionality parity with current C++ and the Modules TS, and to support `#include` translation.
- It would be possible to change the Modules TS "reachability" rule to apply only to entities outside the purview of the module -- that is, we could use the Atom rule ("every semantic effect in the (purview of the) module is exported") for declarations below the *module-declaration* and the Modules TS rule ("only reachable semantic effects are exported") for declarations above the *module-declaration*.

§ References

§ Informative References

[N4719]

Gabriel Dos Reis. [Programming Languages — Extensions to C++ for Modules](https://wg21.link/n4719). URL: <https://wg21.link/n4719>

[P0273R0]

Richard Smith, Chandler Carruth, David Jones. [Proposed modules changes from implementation and deployment experience](https://wg21.link/p0273r0). 12 February 2016. URL: <https://wg21.link/p0273r0>

[P0584R0]

Gabriel Dos Reis. [Module Interface and Preamble](https://wg21.link/p0584r0). URL: <https://wg21.link/p0584r0>

[P0629R0]

Gabriel Dos Reis, Jason Merrill, Nathan Sidwell. [Module interface vs. implementation](https://wg21.link/p0629r0). URL: <https://wg21.link/p0629r0>

[P0713R1]

Daveed Vandevoorde. [Identifying Module Source](https://wg21.link/p0713r1). URL: <https://wg21.link/p0713r1>

[P0775R0]

Nathan Sidwell. [module partitions](https://wg21.link/p0775r0). URL: <https://wg21.link/p0775r0>

[P0795R0]

Simon Brand, Neil Henning, Michael Wong, Christopher Di Bella, Kenneth Benzie. [From Vulkan with love: a plea to reconsider the Module Keyword to be contextual](https://wg21.link/p0795r0). URL: <https://wg21.link/p0795r0>

[P0924R0]

Nathan Sidwell. [Modules: Context-Sensitive Keyword](https://wg21.link/p0924r0). URL: <https://wg21.link/p0924r0>

[P0947R0]

Richard Smith. [Another take on Modules](https://wg21.link/p0947r0). URL: <https://wg21.link/p0947r0>

[P0955R0]

Bjarne Stroustrup. [Modules and macros](https://wg21.link/p0955r0). URL: <https://wg21.link/p0955r0>